Stoquify Enterprise UI/UX Remediation: How to Remove the Gates and Blockers

Date: 2026-08-27

Audience: product sponsors, engineering leadership, finance/control, security, accessibility, quality assurance, and release owners

Source: docs/system-audit/STOQUIFY_ENTERPRISE_UI_UX_REMEDIATION_ROADMAP_2026-08-06.md and what-next/ui-ux/AQSTOQFLOW_UIUX_ENTERPRISE_REMEDIATION_GATE_0_BLOCKER_2026-08-27.md

Purpose

This guide explains, in plain English, what must happen before Stoquify can safely complete the Enterprise UI/UX Remediation Roadmap. The aim is not to make the gates disappear by changing a status label. The aim is to resolve the risks that caused the gates, collect evidence that the fixes work, and obtain the approvals required to release safely.

The most important rule is simple: do not start broad visual modernization until the system can reliably show correct business information, display the correct currency, enforce access correctly, complete visible actions honestly, and support users with accessibility needs.

What the gates mean

A gate is a required checkpoint. It prevents the team from moving to the next phase until the right decision, implementation, test evidence, and reviewer sign-off exist.

The gates are not bureaucratic obstacles. They prevent specific failures:

- Incorrect inventory totals being treated as business truth.
- XAF, XOF, EUR, or USD users seeing the wrong currency presentation.
- Users gaining access to a module they have not purchased or should not use.
- A button claiming an export completed when no file exists.
- A keyboard or screen-reader user being unable to log in or recover from an error.
- A new visual theme breaking a financial or operational workflow.

Current position

The current Gate 0 baseline found 147 page routes, 129 dashboard routes, 410 module surfaces, and 126 uncommitted or new workspace entries. Static TypeScript and lint checks pass, but that does not prove workflow correctness or release readiness.

The current release blockers are:

1. Inventory headline metrics are calculated from the currently visible page of items and hide failures by turning failed reads into empty values.
2. Shared currency formatting still defaults to USD in important utilities.
3. Module entitlement remains in observe mode rather than consistently enforcing access. The current inventory has 293 enforcement candidates and 79 entries classified as missing a permission.
4. The prior simulated customer export has been replaced, but the product still lacks one system-wide contract proving that every enabled action and export has a real result.
5. The active login and registration forms lack the accessible error, hint, and browser-autofill semantics required for a reliable WCAG review.

Step 1: Stabilize the work area

Before the team changes shared UI, routes, access policy, or formatting:

1. Identify which of the 126 workspace changes belong together.
2. Commit each coherent set to a named branch, or move unrelated work to separate worktrees.
3. Record who owns each shared file that is likely to be touched, especially global CSS, authentication components, route configuration, permissions, module entitlement, money-formatting utilities, inventory pages, and test fixtures.
4. Keep a before-and-after git status record for every remediation package.
5. Do not use reset, discard, or broad formatting commands to make the tree appear clean.

Success criteria

- The team can say which change set produced each test result.
- No UI remediation work overwrites another team's active work.
- A defect can be rolled back without undoing unrelated payroll, finance, migration, or data work.

Step 2: Approve the decisions that implementation cannot safely guess

The roadmap calls these decisions ADRs: short written records of a decision, its owner, consequences, and rollback approach. The following decisions require accountable approval before Phase A begins.

Decision	What must be agreed	Why it matters	Accountable reviewers
Inventory KPI meaning	Define inventory cost value, retail potential, estimated margin, low-stock rule, filters, and as-of time.	Prevents a sales-price estimate from being presented as an accounting total.	Inventory product, finance/control, accounting.
Display context	Confirm the server-owned source for locale, currency, timezone, and number format; ban silent USD fallback.	Prevents incorrect money symbols, precision, and date/time presentation.	Platform, finance/control, localization.
Entitlement source and enforcement	Decide which commercial/package record is authoritative and how legacy tenants migrate.	Prevents incorrect access or accidental lockouts.	Security/IAM, billing/package owner, product, support.

Decision	What must be agreed	Why it matters	Accountable reviewers
Canonical routes	Select one official route for duplicate screens and auth variants; define redirects and expiry.	Prevents duplicate workflows, support confusion, and inconsistent analytics.	Product, UX, web platform, support.
Data-state language	Define loading, empty, filtered-empty, partial, stale, unavailable, denied, locked, and not-configured states.	A failed request must never look like a valid zero-data result.	Platform, design system, accessibility, support.
Action and export result	Define the required artifact metadata: ID, scope, filters, row count, timestamp, type/hash, expiry, and download authorization.	A success toast is not proof that an action completed.	Application platform, audit/evidence, security/privacy.
Appearance support	Decide whether dark-only is the temporary supported state or whether dark, light, and system all must work.	The settings screen must never promise themes that product routes do not honour.	Product, design system, accessibility.
Signup and onboarding	Identify legal/security-required signup fields and what can be deferred safely to onboarding.	Simplifies registration without removing consent, anti-abuse, or required company data.	Product, security, legal/compliance, accessibility.
Certification scope	Confirm required browsers, screen readers, viewport sizes, user roles, packages, locales, currencies, and evidence retention.	Gives QA a stable definition of “tested.”	QA/release, accessibility, security, product.
Release authority	Name who can accept evidence and who can accept or reject residual risk.	No implementation team should certify its own finance, accessibility, or security controls.	Executive sponsor and release authority.

How to run the decision process

- Give each decision one owner and a one-page briefing containing the options, affected workflows, recommendation, risk, rollback plan, and required test evidence.
- Hold a decision meeting with the people named in the table.
- Record the answer, not just discussion notes.
- If a decision cannot be made, name the blocker, the person who must resolve it, and a date. Do not silently choose an implementation default.
- Update the roadmap decision log after every approval.

Success criteria

- All ten decisions are approved or have a named external blocker.
- Every affected team knows the one official rule to implement.
- There is a compatible migration and rollback path for legacy users and routes.

Step 3: Create safe, repeatable test data

Build synthetic test organizations. Do not use real customer data and do not reset a shared database.

The fixture set should cover:

- Roles: owner, cashier, inventory controller, purchaser, AP clerk, accountant, external accountant, payroll operator, employee, manager, security administrator, and support.

- Packages: full suite, core-only, inventory, purchasing/AP, finance/accounting, HRIS/payroll, POS, suspended, and read-only.
- Locales and currencies: English/French and XAF, XOF, EUR, USD.
- Business states: active, locked, denied, not configured, empty, filtered empty, partial, stale, error, and success.
- Tenant and location boundaries: at least two organizations and assigned-location restrictions to prove data cannot cross a boundary.

Each fixture needs a stable identifier, a safe setup procedure, a cleanup procedure, redaction rules for screenshots/logs, and an expected permission result.

Success criteria

- A test can reliably log in as each approved role.
- The same test can prove both allowed access and denied access.
- Screenshots and logs contain no secrets or real customer data.
- A test failure can be reproduced without manually repairing data.

Step 4: Close the Phase A release blockers

A. Make inventory metrics trustworthy

Move dashboard headline calculations to a server-owned inventory summary. The summary must calculate across the full authorized scope, not the rows visible in the item table.

Every returned metric must include:

- the organization, location, and filter scope;
- the currency;
- an as-of timestamp;
- a source status: complete, partial, stale, or unavailable;
- clear labels that distinguish cost value, retail potential, and estimated margin.

Use each item's reorder point or an approved organization policy for low-stock logic. Do not use a hidden global < 10 rule.

Test the same filters at page sizes of 10, 25, 50, and 100. The headline figures must be identical. If the source fails, show an unavailable or partial state; never show a convincing zero.

B. Establish one currency and locale contract

Create a server-derived DisplayContext carrying organization and location scope, locale, currency, timezone, and number-format rules. Pass it to financial read models, UI formatters, notifications, and exports.

Then migrate the highest-risk business surfaces first: inventory, purchase orders, finance/accounting, payments, POS, payroll, reports, notifications, and exports. Add a CI check that rejects new raw dollar symbols, hard-coded USD, en-US, and money toFixed() formatting unless the line is an approved fixture or selector.

C. Turn entitlement observation into safe enforcement

Use one capability decision everywhere. It must combine user permission, package entitlement, organization state, location scope, and intended action such as read, edit, approve, or export.

Roll out in stages:

1. Keep observe mode and record every would-block event.
2. Classify and fix missing permissions and mismatches.
3. Test a small pilot package/tenant cohort in enforce mode.
4. Monitor denial, mismatch, bypass, and support-contact rates.
5. Expand only when the cohort is stable; retain an immediate switch back to observe mode for that cohort.

Server authorization remains enforced throughout. Returning to observe mode must not create a security bypass.

D. Make every visible action honest

Inventory all visible buttons, menu items, keyboard commands, and automated actions. Every one must be one of these:

- executable and connected to a real outcome;
- disabled with an understandable reason;
- locked because the tenant lacks a module/package;
- hidden because the action should not be discoverable.

For an export, success requires a real artifact with file type, artifact ID, scope, filters, row count, timestamp, expiry, authorized download location, and audit correlation. Remove any simulated completion message. The existing real customer export is a useful reference, but it is not proof that the rest of the product is covered.

E. Repair public and authentication accessibility

For active login and registration flows:

- Attach labels to all controls.
- Use browser autocomplete values for identity and password fields.
- Set aria-invalid on invalid controls.
- Connect error and hint text with aria-describedby.
- Announce validation and server errors through a safe live region.
- Move focus to the first invalid field after submit.
- Keep contrast, focus visibility, keyboard order, 320-pixel layouts, and 400-percent zoom usable.

Run automated axe checks and keyboard tests in English and French. Then have a qualified person complete the flows with NVDA and VoiceOver. Automated scans alone cannot close this gate.

Phase A success criteria

- No page-sliced or silently failed inventory headline is shown as a complete business total.
- No unapproved currency fallback is present on risk-tier business surfaces.

- UI visibility and server authorization agree for tested role/package/location combinations.
- No enabled export/action claims success without a real result.
- Required public/auth journeys have no critical or serious automated accessibility violations and pass human assistive-technology review.

Step 5: Build the shared platform rules before normalizing every module

After Phase A, establish reusable foundations:

1. Route registry: one typed source listing canonical routes, aliases, owner, module, permission/capability, states, and retirement date.
2. State registry: one shared way to render loading, empty, partial, stale, error, denied, locked, and not-configured states.
3. Visual token contract: semantic colors, typography, spacing, surfaces, contrast rules, and approved exceptions. No new ad-hoc theme islands.
4. Certification manifest: a machine-readable map from routes to required fixtures, browsers, locales, viewports, states, accessibility tests, and visual evidence.
5. Resumable onboarding: users can complete setup over time without losing progress or bypassing consent/security rules.

Do not rebuild modules before these rules are stable. Otherwise the team will create six versions of the same problem.

Step 6: Normalize modules in controlled waves

Use the shared rules to update one module wave at a time:

1. Inventory.
2. Purchasing and AP.
3. Finance, accounting, reconciliation, and close.
4. HRIS and payroll.
5. Settings and administration.
6. POS within the approved desktop/tablet boundary.

For each wave, verify the actual user job, data truth, permission/entitlement behavior, evidence trail, currency/locale, empty/error states, keyboard path, responsive layout, exports, and domain controls. Preserve existing business logic unless a defined contract requires a change.

Do not declare a wave complete merely because it looks visually consistent.

Step 7: Certify, roll out, and decide release

When all waves are complete:

1. Run automated functional, route, permission, export, accessibility, visual, type, lint, policy, build, and regression checks.
2. Complete human finance/control, security/privacy, accessibility, localization, support, and domain workflow validation.
3. Verify performance and production observability using representative data; do not use local development timings as production proof.
4. Run a staged tenant rollout with clear monitoring, support communication, and a tested rollback procedure.
5. Preserve all evidence, including failed runs and justified reruns.
6. Have the independent assurance authority issue the final decision.

What should happen next

The next practical action is a Gate 0 decision workshop. The sponsor should appoint the reviewers listed in Step 2 and ask them to approve the ten decisions or name a blocker owner and due date.

In parallel, QA can prepare the synthetic-fixture design and engineering can map the 410 module surfaces, but neither should invent financial formulas, commercial entitlements, canonical routes, legal signup fields, or release policy without approval.

Definition of a genuinely cleared gate

A gate is cleared only when all four are true:

1. The rule is explicitly agreed and owned.
2. The code implements that rule without a hidden fallback.
3. Automated tests and evidence prove the expected behavior and failure behavior.
4. The named independent reviewer accepts the evidence and rollback plan.

Passing lint or type checking is useful, but it satisfies only a small part of item 3. It never replaces the other three conditions.

Supporting evidence

- what-next/ui-ux/AQSTOQFLOW_UIUX_ENTERPRISE_REMEDIATION_GATE_0_BLOCKER_2026-08-27.md
- docs/system-audit/STOQUIFY_ENTERPRISE_UI_UX_SYSTEM_AUDIT_2026-08-06.md
- docs/system-audit/STOQUIFY_ENTERPRISE_UI_UX_REMEDIATION_ROADMAP_2026-08-06.md
- what-next/ui-ux/enterprise-remediation-roadmap/